```
    private readonly ServiceA _a;                                    ❶
    public HeadersMiddleware(RequestDelegate next, ServiceA a)       ❶
    {                                                                ❶
        _next = next;                                                ❶
        _a = a;                                                      ❶
    }
    public async Task Invoke(
        HttpContext context, ServiceB b, ServiceC c)                 ❷
    {
        // use services a, b, and c
        // and/or call _next.Invoke(context);
    }
}
```

❶ You can inject additional services in the constructor. These must be singletons.

❷ You can inject services into the Invoke method. These may have any lifetime.

WARNING ASP.NET Core creates the middleware only once for the lifetime of your app, so any dependencies injected in the constructor must be singletons. If you need to use scoped or transient dependencies, inject them into the `Invoke` method.

In addition to cross-cutting concerns, a good use for middleware is creating simple handlers with as few dependencies as possible that respond to a fixed URL, similar to the `Use` extension method you learned about in section 31.1.3. These simple handlers can be dropped into multiple applications, regardless of how the app's routing is configured.

So-called *well-known Uniform Resource Identifiers* (URIs) are a good use case for these simple middleware handlers, such as the security.txt well-known URI (https://www.rfc-editor.org/rfc/rfc9116) and the OpenID Connect URIs (http://mng.bz/wvj2). These handlers always respond to a single path, so they can neatly encapsulate all the logic without risk of interfering with any other routing configuration.